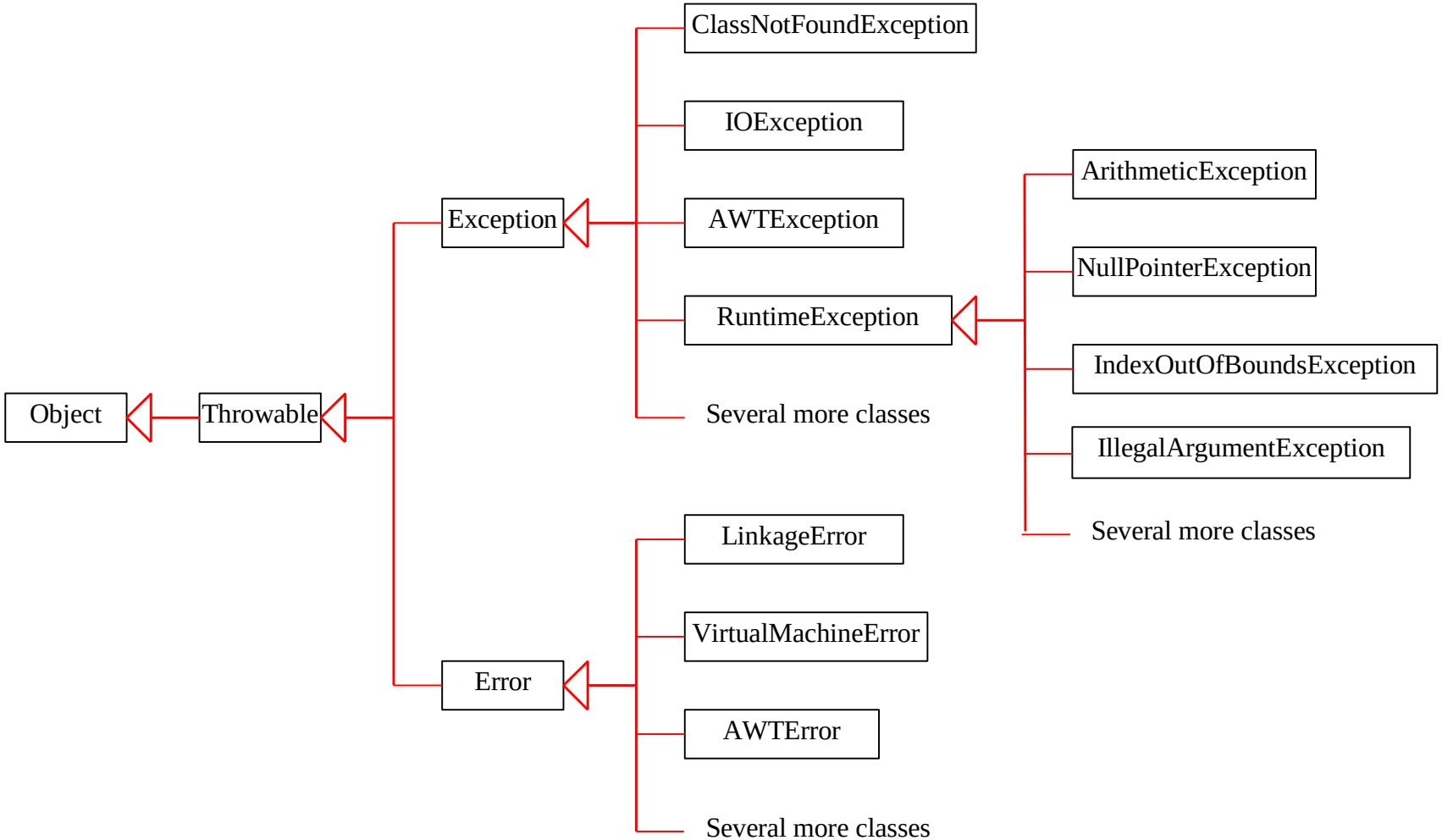
Exception Handling

Contents

- Exception Handling Keywords
- Java Multi Catch block
- The finally block
- Various possible combination of try catch finally
- Customised Exception

Hierarchy of Java Exception classes



Java Exception Handling Keywords

There are 5 keywords used in java exception handling.

- **Try** : Java try block is used to enclose the code that might throw an exception. It must be used within the method. Java try block must be followed by either catch or finally block.

- **Catch** : Java catch block is used to handle the Exception. It must be used after the try block only. We can use multiple catch block with a single try.

- **Finally** : Executes whether or not an exception is thrown in the corresponding try block or any of its corresponding catch blocks.

- **Throw** : You can throw an exception, either a newly instantiated one or an exception that you just caught, by using the throw keyword.

- **Throws** : If a method does not handle a checked exception, the method must declare it using the throws keyword.

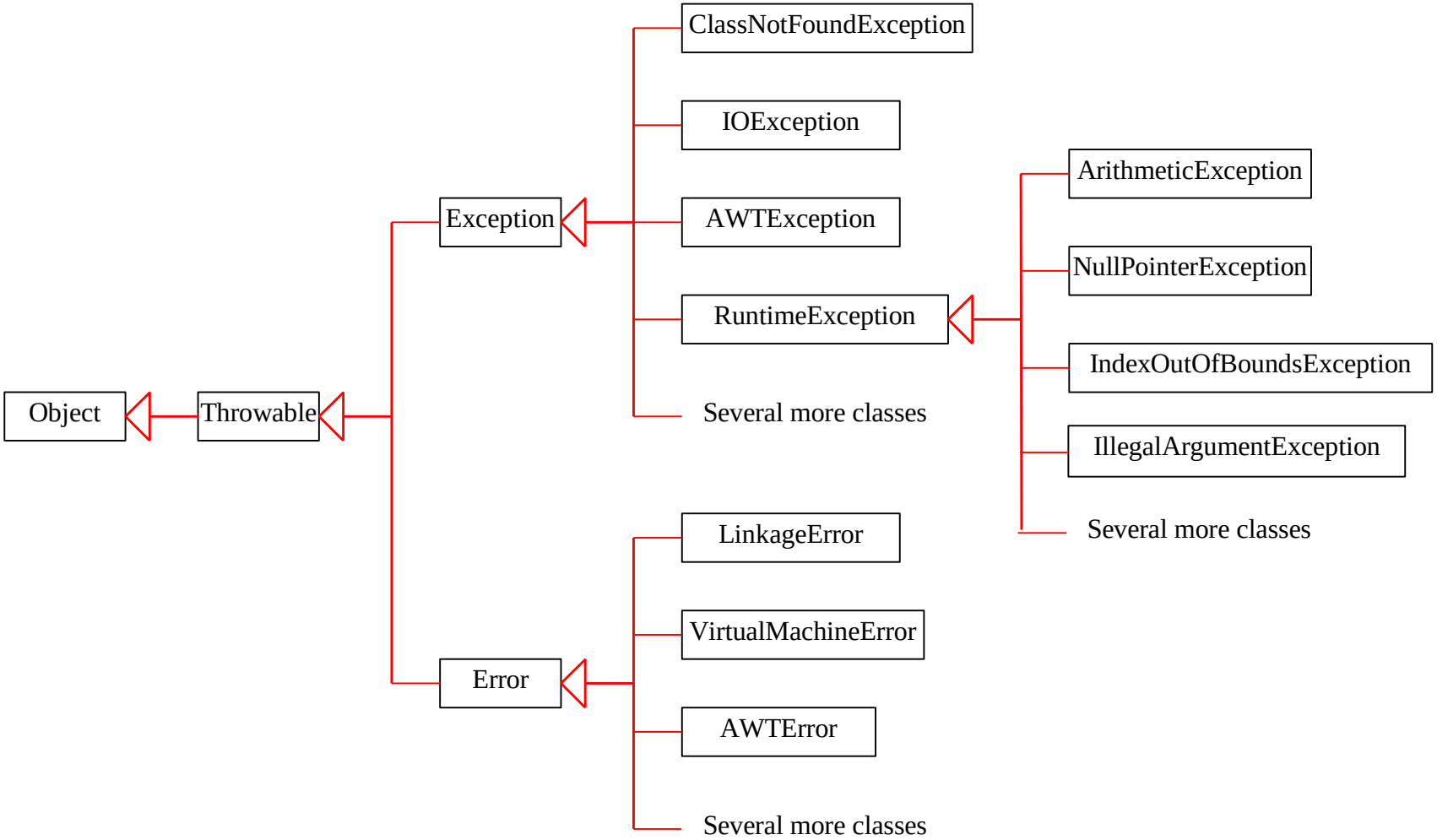
Checked Exceptions

- Inherit from class Exception but not from RuntimeException
- Compiler enforces catch-or-declare requirement
- Compiler checks each method call and method declaration

e.g. IOException, SQLException etc.

Unchecked Exceptions

- Inherit from class RuntimeException or class Error
- Compiler does not check code to see if exception caught or declared
- If an unchecked exception occurs and not caught
  - Program terminates or runs with unexpected results
- Can typically be prevented by proper coding


Object
Throwable
Exception
Error
ClassNotFoundException
IOException
AWTException
RuntimeException
Several more classes
ArithmeticException
NullPointerException
IndexOutOfBoundsException
IllegalArgumentException
Several more classes
LinkageError
VirtualMachineError
AWTError
Several more classes

```
try{
Risky code;
}
catch(Exception e)
{
Handling code;
}
```

```
try{
Stmt 1;
Stmt 2;
Stmt 3;
}
catch(ArithmeticException e)
{
Stmt 4;
}
Stmt 5;
```

Case 1: 1,2,3,5; NT
Case 2: 1,4,5; NT
Case 3: 1; AT

Java Multi Catch block

```
public class Test4{
public static void main(String args[]){
 try{
   int a[]=new int[5];
   a[2]=30/0;
  }
  catch(ArithmeticException e){
  System.out.println("task1 is completed");}

  catch(ArrayIndexOutOfBoundsException e)
{System.out.println("task 2 completed");}

  catch(Exception e){System.out.println("common task completed");}
  System.out.println("rest of the code...");
 }
}
```

- All catch blocks must be ordered from most specific to most general i.e. catch for ArithmeticException must come before catch for Exception .

The finally block

- The finally block follows a try block or a catch block. A finally block of code always executes, irrespective of occurrence of an Exception.
- Using a finally block allows you to run any cleanup-type statements that you want to execute, no matter what happens in the protected code.
- A finally block appears at the end of the catch blocks and has the following syntax:

```
try
{ //Risky code
}
catch(ExceptionType1 e1){
//Catch block }

 catch(ExceptionType2 e2){
//Catch block}

finally {
 //The finally block always executes.
 }
```

Example of finally

```
class Finally{
public static void main(String args[]){
try{
int d=25/5;
System.out.println(d);
}
catch(NullPointerException e){
System.out.println(e);
}
finally {
System.out.println("Finally block is executed");
}

System.out.println("Rest of the code");
}
}
```

```
5
Finally block is executed
Rest of the code
```

Example of finally

```
class Finally{
public static void main(String args[]){
try{
int d=25/0;
System.out.println(d);
}
catch(NullPointerException e){
System.out.println(e);
}
finally {
System.out.println("Finally block is executed");
}

System.out.println("Rest of the code");
}
}
```

```
Finally block is executed
Exception in thread "main"
java.lang.ArithmeticException: / by zero
        at Finally.main(Finally.java:4)
```

Example of finally

```
class Finally{
public static void main(String args[]){
try{
int d=25/0;
System.out.println(d);
}
catch(ArithmeticException e){
System.out.println(e);
}
finally {
System.out.println("Finally block is executed");
}

System.out.println("Rest of the code");
}
}
```

```
java.lang.ArithmeticException: / by zero
Finally block is executed
Rest of the code
```

Various possible combination of try catch finally

```
try{

}
catch(X e)
{

}
```

```
try{  }

catch(X e){
}
catch(Y e){
}
```

```
try{  }

catch(X e){
}
catch(X e){
}
```

CE: Exception has already been caught.

```
try{  }

catch(X e){
}
finally{
}
```

Various possible combination of try catch finally

```
try{

}
finally
{

}
```

```
try{  }

catch(X e){
}
try{ }
catch(Y e){
}
```

```
try{  }

catch(X e){
}
try{ }
finally{
}
```

```
try{  }
```

CE: try without catch or finally.

Various possible combination of try catch finally

```
try{ }
S.O.P("Hi");
catch(X e)
{

}
```

CE: 1. try 2. catch

```
try{ }
catch(X e)
{

}
S.O.P("Hi");
catch(Y e){
}
```

CE: catch

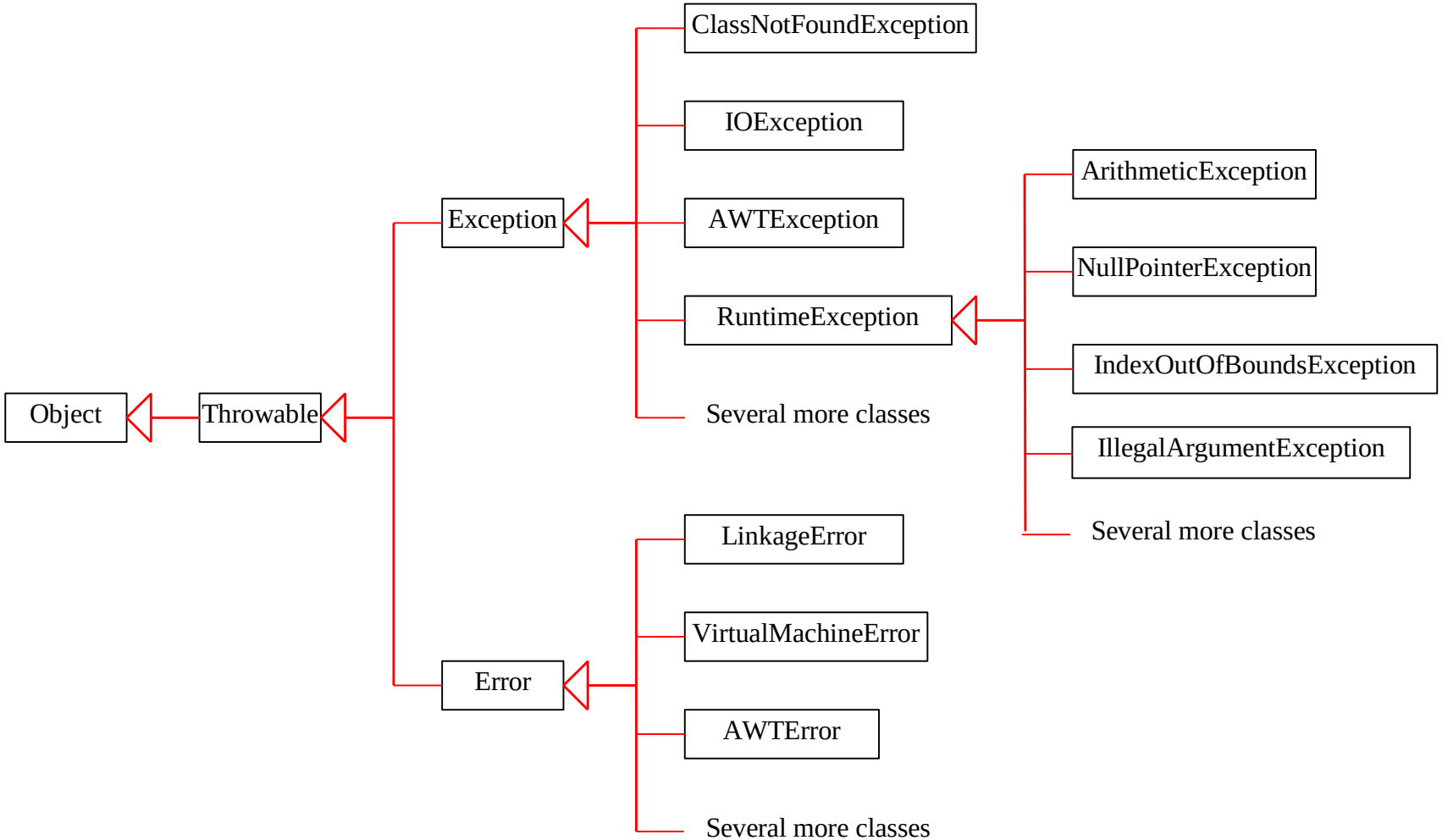
```
try{

try{  }
catch(X e){
}
catch(Y e)
{ }
```

```
try{
try{  }
}
catch(X e)
{
}
```

CE: try without catch

Hierarchy of Java Exception classes



Checked Exceptions vs. Unchecked Exceptions

- RuntimeException, Error and their subclasses are known as unchecked exceptions.
- All other exceptions are known as checked exceptions - the compiler forces the programmer to check and deal with the exceptions.

Declaring Exceptions

- Every method must state the types of checked exceptions it might throw. This is known as declaring exceptions.
- public void myMethod() throws IOException
- public void myMethod() throws IOException, ServletException

Throwing Exceptions

- If we want to throw an exception manually or explicitly, for this, Java provides a keyword throw.
- Throw in Java is a keyword that is used to throw a built-in exception or a custom exception explicitly or manually. Using throw keyword, we can throw either checked or unchecked exceptions in Java programming.,
- throw new TheException();
- TheException ex = new TheException();
  throw ex;

Without throw

```
class Test6{

public static void main(String args[]){

System.out.println(10/0);

}
}
```

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
        at Test6.main(Test6.java:3)
```

With throw

```
class Test6{
public static void main(String args[]){
throw new ArithmeticException("divide by Zero
Explicitly");
}
}
```

```
Exception in thread "main"
java.lang.ArithmeticException: divide by zero
explicitly
        at Test6.main(Test6.java:3)
```

With throw

```
import java.util.Scanner;
class Test{
public static void main(String args[]){
int a, b;
Scanner sc=new Scanner(System.in);
a=sc.nextInt();
b=sc.nextInt();
int c=a/b;
if(c>=0 || c<0){
System.out.println(c);}
else{
throw new ArithmeticException();}
}
}
```

```
6
5
1
Java Test
-15
3
-5
Java Test
50
0
Exception in thread "main"
java.lang.ArithmeticException: / by
zero
        at Test.main(Test.java:8)
```

Example

```
class Test7{
static ArithmeticException e=new ArithmeticException();
public static void main(String args[]){
throw e;
}
}
```

```
Exception in thread "main"
java.lang.ArithmeticException
        at Test7.<clinit>(Test7.java:2)
```

Example

```
class Test9{
public static void main(String args[])
{
System.out.println(10/0);
System.out.println("Hello");
}
}
```

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
        at Test9.main(Test9.java:3)
```

Example

```
class Test10{
public static void main(String args[]){
throw new ArithmeticException("/ by zero");
System.out.println("Hello");
}
}
```

```
Test10.java:4: error: unreachable statement
System.out.println("Hello");
^
1 error
```

Example

```
class Test0{
public static void main(String args[]){
throw new Test0();
}
}
```

```
Test0.java:3: error: incompatible types: Test0
cannot be converted to Throwable
throw new Test0();
^
1 error
```

Example

```
class Test01 extends RuntimeException{
public static void main(String args[]){
throw new Test01();
}
}
```

```
Exception in thread "main" Test01
        at Test01.main(Test01.java:3)
```

Example

```
class Tes1 {
public static void main(String args[]){
throw new Exception();
}
}
```

```
Tes1.java:3: error: unreported exception Exception;
must be caught or declared to be thrown
throw new Exception();
^
1 error
```

Example

```
class Ts1 {
public static void main(String args[]){
throw new Error();
}
}
```

```
Exception in thread "main" java.lang.Error
        at Ts1.main(Ts1.java:3)
```

Example of checked Exception

```
import java.io.*;
class Example {
  public static void main(String args[])  {
        FileInputStream fis = new FileInputStream("D:/myfile.txt");
        int k;
        while(( k = fis.read() ) != -1)  {
              System.out.print((char)k);
        }
        fis.close();  } }
```

```
Example.java:6: error: unreported exception FileNotFoundException; must be caught
or declared to be thrown
      fis = new FileInputStream("D:/myfile.txt");
            ^
Example.java:9: error: unreported exception IOException; must be caught or declared
to be thrown
      while(( k = fis.read() ) != -1)
                  ^
Example.java:14: error: unreported exception IOException; must be caught or
declared to be thrown
      fis.close();
           ^
3 errors
```

Example of checked Exception

```
import java.io.*;
class Example {
   public static void main(String args[])  throws IOException
  {
      FileInputStream fis =  new FileInputStream("D:/myfile.txt");
                int k;
                while(( k = fis.read() ) != -1)
                  {
                            System.out.print((char)k);
                }
                fis.close();
  }
 }
```

Example

Wrong

```
class Test throws Exception {
Test() throws Exception
{
}
public void m1() throws Exception{
}
}
```

Example

```
class Test
{
public static void main(String args[]) throws Test{
}
}
```

```
Test.java:2: error: incompatible types: Test cannot be
converted to Throwable
public static void main(String args[]) throws Test{
                                               ^
1 error
```

Example

```
class Test extends RuntimeException{
public static void main(String args[]) throws Test
{
throw new Test();
}

}
```

Re-throw Exception

```
class TestThrow1{
public static void main(String args[]) {
try{
System.out.println(10/0);
}
catch(ArithmeticException e)
{
throw new ClassCastException();
}
}
}
```

```
Exception in thread "main"
java.lang.ClassCastException
        at TestThrow1.main(TestThrow1.java:7)
```

Re-throw Exception

```
class TestThrow{
static void demo(){
try{
throw new NullPointerException();
  }
catch(NullPointerException e){
System.out.println("Caught inside demo");
throw new ArithmeticException();
}
}
public static void main(String args[]) {
try{
demo();
}catch(ArithmeticException e)
{
System.out.println("Re-Caught :" + e);}
}
}
```

```
Caught inside demo
Re-Caught :java.lang.ArithmeticException
```

Difference between throw and throws

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions public void method() throws IOException, SQLException.

Customized Exception

```
class MyCustomException extends Exception
{

}

public class Test
{
  public static void main(String args[])
    {
       try
       {
        throw new MyCustomException();
       }
       catch (MyCustomException ex)
       {
          System.out.println("Caught the exception");
          System.out.println(ex);
       }
System.out.println("rest of the code...");
    }
}
```

```
Caught the exception
MyCustomException
rest of the code...
```

Overriding Rule

- **If the superclass method does not declare an exception**
  - If the superclass method does not declare an exception, subclass overridden method cannot declare the checked exception but it can declare unchecked exception.

- **If the superclass method declares an exception**
  - If the superclass method declares an exception, subclass overridden method can declare same, subclass exception or no exception but cannot declare parent exception.

Example

```
import java.io.*;
class Parent{
   void msg() {
    System.out.println("parent method");
    }
}

public class Test extends Parent{

  void msg() throws IOException {
    System.out.println("Test Exception Child");
  }

  public static void main(String args[]) {
   Parent p = new Test();
   p.msg();
  }
}
```

```
Test.java:10: error: msg() in Test cannot
override msg() in Parent
  void msg() throws IOException {
       ^
  overridden method does not throw
IOException
1 error
```

Example

```
import java.io.*;
class Parent{
   void msg() {
    System.out.println("parent method");
    }
}

public class Test extends Parent{

  void msg() throws ClassCastException {
    System.out.println("Test Exception Child");
  }

  public static void main(String args[]) {
   Parent p = new Test();
   p.msg();
  }
}
```

Example

```
import java.io.*;
class Parent{
  void msg() throws ArithmeticException {
    System.out.println("parent method");
  }
}

public class Test extends Parent{
  void msg() throws Exception {
    System.out.println("child method");
  }

  public static void main(String args[]) {
   Parent p = new Test();
   p.msg();
   }
    }
```

```
Test.java:9: error: msg() in Test cannot override
msg() in Parent
  void msg()throws Exception {
       ^
  overridden method does not throw Exception
1 error
```

Example

```
import java.io.*;
class Parent{
  void msg() throws ArithmeticException {
    System.out.println("parent method");
  }
}

public class Test extends Parent{
  void msg() throws ArithmeticException {
    System.out.println("child method");
  }

  public static void main(String args[]) {
   Parent p = new Test();
   p.msg();
   }
    }
```

child method

Example

```
import java.io.*;
class Parent{
  void msg() throws ArithmeticException {
    System.out.println("parent method");
  }
}

public class Test extends Parent{
  void msg() throws ArithmeticException {
    System.out.println("child method");
  }

  public static void main(String args[]) {
   Parent p = new Test();
   p.msg();
   }
    }
```

child method

THANK YOU